(e) **T** **F** Checking if a string of length k exists in a hash table takes worst-case $O(k)$ time.

Solution: False. Hash table operations are expected time operations.

(f) T F [3 points] In a hash table maintained with cuckoo hashing, the worst-case running time to search for an element is $O(1)$.

Solution: True. There are only two places needed to be searched.

(g) **T** **F** [3 points] Consider a hash table with open addressing and linear probing. Any table resulting from insertions and deletions into an empty table can also be created just by inserting into an empty table.

Solution: False. There is an example of this in problem set 3.

(c) [5 points] Suppose we have a hash table, with collisions resolved by chaining, and we implement table doubling to resize the table but do not change the hash function. Why is this a bad idea?

Table doubling is employed to distribute data over a larger number of slots, and therefore reduce the length of chains. However, if we double the table without modifying the hash function, we will only hash elements to the first half of the table, leaving any existing chains unaffected. In fact, the performance of all future operations will be exactly the same as if the table doubling never happened. Finally, we end up consuming more memory to house hash cells which will never become occupied.

(d) **T** **F** Given an array of n key-value pairs, one can construct a hash table mapping keys to values in **expected** $O(n)$ time by inserting normally into the hash table one at a time, where collisions are resolved via chaining. If the pairs have unique keys, it is possible to construct the same hash table in **worst-case** $O(n)$ time.

Solution: True. Because keys are unique, we do not have to check for duplicate keys in chain during insertion, so each insert can take worst-case $O(1)$ time.

Problem 4. [6 points] **Expect the Unexpected** (2 parts)

Suppose you have a hash table of size m, storing n keys, where $m = \Theta(n)$, and collisions are resolved by chaining. Assume the simple uniform hashing assumption.

(a) [3 points] What is the **expected** running time of SEARCH(x)?

Solution: It is given than $m = \Theta(n)$, so in this hash table a search has expected runtime $\Theta(1)$.

Common Mistake: An answer of $\Theta(\frac{n}{m})$ can be simplified further because it was specified that $m = \Theta(n)$.

(f) **T** **F** Given a weighted connected undirected graph $G = (V, E)$ containing exactly $|V| - 1$ edges, one can solve weighted Single-Source Shortest Paths from any $s \in V$ in $O(|V|)$ time.

Solution: True. A connected graph with exactly $|V| - 1$ edges is a tree and contains no undirected cycles. Run breadth-first search from s in $O(|V|)$ time to direct every edge away from s, and then run DAG relaxation on the resulting DAG from s to compute single-source shortest paths, also in $O(|V|)$ time.

(g) **T** **F** Depth-first search solves single-source shortest paths in an unweighted, directed graph $G = (V, E)$ in $O(|V| + |E|)$-time.

Solution: False. A complete graph on three vertices is a counter example.

(h) **T** **F** Given a connected weighted directed graph having positive integer edge weights, where each edge weight is at most k, we can compute single source shortest paths in $O(k|E|)$ time.

Solution: True. Replace each edge (a, b) with weight w with a directed unweighted path from a to b, and run BFS from the source.

(f) **T** **F** Given a directed graph $G = (V, E)$, run breadth-first search from a vertex $s \in V$. While processing a vertex u, if some $v \in \text{Adj}^+(u)$ has already been processed, then G contains a directed cycle.

Solution: False. BFS can't be used to find directed cycles. A counterexample is $V = \{s, a, b, t\}$ and $E = \{(s, t), (s, a), (a, b), (b, t)\}$. Running BFS from s will first process vertices in levels $\{s\}$, then $\{a, t\}$, then $\{b\}$. When processing vertex b, vertex $t \in \text{Adj}^+(b)$ has already been processed, yet G is a DAG.

(g) **T** **F** Given a weighted simple directed graph $G = (V, E)$, where every vertex connects to at least $|V|/2$ edges and every edge has strictly negative weight, one can find a maximum-weight directed path from vertex $s \in V$ to $t \in V$ in linear time.

Solution: True. Negate all edge weights so all weights are positive, and run Dijkstra's algorithm from s to t in time $O(|V| \log |V| + |E|)$. There are $|E| = \Omega(|V|^2)$ edges in this graph, so Dijkstra's algorithm runs in linear time.

(g) **T** **F** Run Bellman-Ford on a weighted graph $G = (V, E, w)$ from a vertex $s \in V$. If there is a **witness** $v \in V$, i.e., $\delta_{|V|}(s, v) < \delta_{|V|-1}(s, v)$, then v is on a negative-weight cycle of G.

Solution: False. A witness is only guaranteed to be **reachable** from a negative-weight cycle; it may not actually be **on** a negative-weight cycle.

(k) **T** **F** Given a weighted undirected graph $G = (V, E)$. For any starting node s, in order to find a path of least total weight from s to any node in G, it suffices to apply Bellman-Ford algorithm with only $|V|/2$ iterations.

Solution: False, it is actually $|V| - 1$ iterations.

(n) **T** **F** If a directed graph G contains negative-weight edges but no negative-weight cycles, then the Bellman-Ford algorithm can be used to compute the shortest path between any two nodes.

Solution: True. Bellman-Ford can handle a graph with negative weight edges.